

Assignment 21: Javascript

Question1:

[Answer1\)](#)

Question2:

Answer2):

Part A)

JavaScript is **single-threaded**, meaning it has **only one call stack** and executes **one line at a time**. If JavaScript executed everything asynchronously by default, then:

- Instructions would run **out of order**
- Variables would become unpredictable
- Normal logic (calculations, loops, assignments, conditions) would break
- You could never rely on the previous line finishing before the next starts

Therefore, JavaScript is synchronous by default so that code executes in a **predictable, sequential order**.

Then how do async tasks work?

JavaScript **itself is synchronous**, but the **browser or Node.js environment** provides:

- Web APIs (setTimeout, fetch, DOM events)
- Promises / async-await
- Callback queues
- Event loop

Async tasks are **delegated** to Web APIs → placed in callback queue
→ executed **later** when the call stack becomes empty.

Simple Code Snippet to Prove JavaScript Is Inherently Synchronous

[Example:](#)

Code Snippet Showing Sync VS Async Behavior:

[Example:](#)

Question3)

Answer3)

Question 4) What are the different states of a Promise, and how do they transition?

Answer 4) The lifecycle consists of promises consists of 4 stages

- 1) **Pending:** The promise is still running and has not produced a result.
- 2) **Resolved:** The promise has completed successfully with a value (fulfilled).
- 3) **Rejected:** The promise has failed with an error or reason.
- 4) **Settled:** The promise has finished running and is either resolved or rejected (no longer pending).

[Example for Promises:](#)

Question 5) How does the JavaScript event loop handle Promises differently from setTimeout?

Answer 5)

1. Promises → Microtask Queue

- When a Promise resolves (.then, .catch, .finally), its callbacks go to the **microtask queue**.
- **Microtasks run immediately after the current call stack finishes, before any macrotasks.**
- So Promises have **higher priority**.

2. setTimeout → Macrotask Queue

- setTimeout callbacks go to the **macrotask (callback) queue**.
- A macrotask runs **only after all microtasks are completed**.
- Even if setTimeout(fn, 0) is used, it runs *after* microtasks.

[Example:](#)